

0.54.2. Trois lettres, trois carrés — doublé, deux positions

Les trois carrés du 0.54, puis celui du D : quatre lettres.

1. Ce qu'il faut comprendre

C'est le 0.54 et le 0.54.1 réunis : les trois carrés, puis le quatrième.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Carre>` → [0.47.3. Carre — les réglages d'un carré sous un nom](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <carre>` → [0.33.3. La fonction carre\(\) — tourner en carré](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1442 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage — la mise en route de la console
carre()	440 o	← main() — une fonction fournie par le langage
main()	131 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases — attend de pouvoir écrire une case
AdresseCase	10 o	← poser(), effacer() — trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer() — efface des cases
CopierTuiles	18 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	704 o	← CopierTuiles — la police entière

Lettres gravées : toutes (poser() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

poser()	× 4	52 o sur place, dans carre() + AdresseCase (10 o)
effacer()	× 3	45 o sur place, dans carre() + AdresseCase (10 o), EffacerCases (12 o)
image()	× 4	12 o sur place, dans carre(), main() + AttendreImage (28 o)

Tes fonctions appelées :

carre()	× 4	124 o sur place, dans main()
---------	-----	------------------------------

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <Carre>      // les sept réglages d'un carré sous un seul nom
#include <ALPHABET>    // les lettres de la police : ALPHABET[0] est le A
#include <carre>       // une tuile qui tourne en carré
```

2. UNE VARIABLE — RONDE

```
Carre ronde = { 15, 8, ALPHABET[2], 2, 1, 250, 2 }; // le C
```

3. UNE VARIABLE — BAS

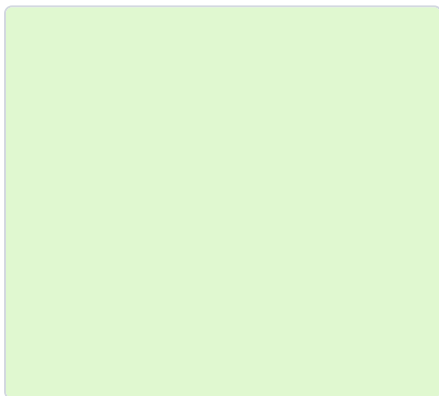
```
Carre bas = { 15, 14, ALPHABET[3], 2, 1, 250, 1 }; // le D
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

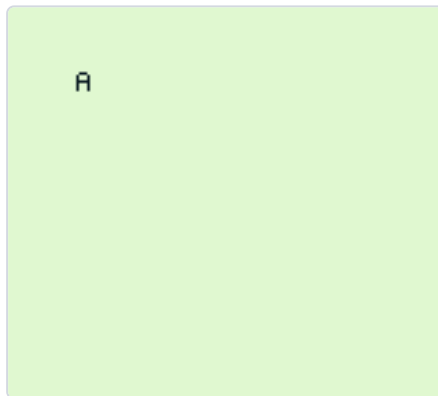
```
int main() {  
    carre(4, 4, ALPHABET[0], 1, 1, 250, 1); // le A  
    carre(9, 9, ALPHABET[1], 3, -1, 100, 1); // le B  
    carre(ronde); // le C  
    carre(bas); // le D  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

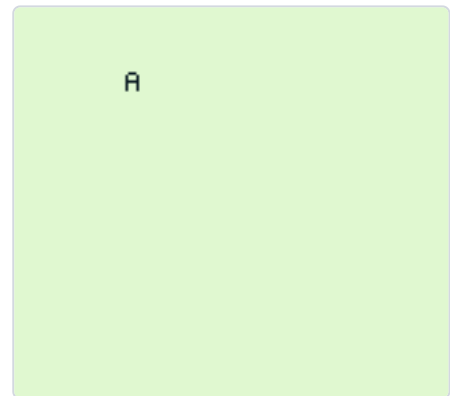
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



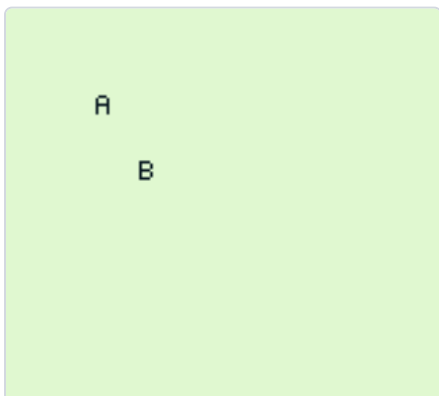
la toute première image — la console vient de démarrer — image 1



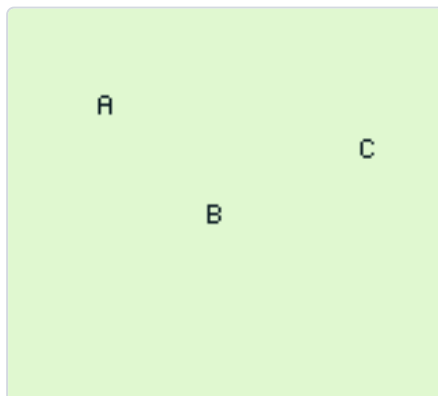
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Le A, le B, le C, puis le D font leur carré.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le A finit en (4, 4) (4,4)

✓ le B finit en (9, 9) (9,9)

✓ le D finit en (15, 14) (15,14)

Ensuite : **0.54.3. La fonction `vitesse()` — régler la vitesse des déplacements** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++